Nombres: Samuel Alejandro Casaos Esquivel Maximiliano Romero García Josue Emiliano Diaz García	**Matrícula**: 2226784 2226867 2226766
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: DISEÑO ARQUITECTURAL DEL SISTEMA
Fecha: 13/10/2026	
Bibliografia (Formato APA)	

Contenido

1. Introducción y Contexto del Negocio 2
2. Etapa 1: Preparación del Caso y Selección de Casos de Uso 2
3. Etapa 2: Elección del Patrón de Lógica de Negocio 3
4. Etapa 3: Mapa de Dominio (Domain-Driven Design) 4
5. Etapa 4: Service Layer (Capa de Aplicación) 4
6. Etapa 5: Persistencia y Transacciones (Repository y Unit of Work) 5
7. Etapa 6: Segregación de Comandos y Consultas (CQRS Básico) 5
8. Etapa 7: Revisión de la Propuesta con Principios SOLID 6
9. Etapa 8: Artefactos Arquitectónicos y Diagramas Oficiales 7
9.1. Diagrama de Clases UML 7
9.2. Diagramas de Secuencia 7
9.3. Architecture Decision Records (ADRs) Registrados 8
9.4. Evidencia del Código Base Implementado en C++ (Capa por Capa) 8
10. Etapa 9: Matriz de Trazabilidad y Validación 11
11. Bitácora Obligatoria de Uso de Inteligencia Artificial 12
12. Conclusiones y Defensa 12
12.1. Conclusiones Generales del Proyecto 12
12.2. Conclusiones Individuales 13
12.3. Respuestas Clave para la Defensa en Clase (Semana del 12 al 16 de Octubre) 13

1. Introducción y Contexto del Negocio

El presente informe documenta la transformación del análisis de requerimientos de negocio de la Unidad 2 en una arquitectura de software robusta, desacoplada y formal para EduTech Academy. El sistema aborda el ciclo integral de gestión académica: preinscripción, confirmación de pagos, administración de grupos, evaluación del desempeño y acreditación formal mediante certificados y constancias.

En el diagnóstico operativo (AS-IS), la institución determinaba la elegibilidad de certificación mediante un cruce manual en hojas de cálculo de Excel entre las listas de asistencia y las calificaciones. Este método manual provocaba discrepancias frecuentes, quejas de estudiantes, demoras superiores a 24 horas y falta de trazabilidad en autorizaciones críticas (cambios de grupo, cancelaciones y correcciones de calificaciones).

El objetivo arquitectural (TO-BE) consiste en diseñar e implementar un núcleo de lógica de negocios desacoplado de la persistencia e interfaz, garantizando cero discrepancias, validación en tiempo real y el estricto cumplimiento de invariantes institucionales mediante el patrón Domain Model, Domain-Driven Design (DDD), principios SOLID y separación CQRS.

2. Etapa 1: Preparación del Caso y Selección de Casos de Uso

A partir de las 20 Historias de Usuario desarrolladas en la Unidad 2, se seleccionaron los 5 casos de uso más representativos del proceso principal. Estos casos abarcan el ciclo de vida completo y contrastan flujos normales frente a excepciones rigurosas:

ID	Caso de Uso	Actor	Reglas Relacionadas	Resultado Esperado
CU-01	Confirmar Inscripción por Pago o Anticipo	Estudiante / Sistema	RN-01: Anticipo mín. 50% RN-17: Plazo máx. 3 días hábiles RN-10: Cupo disponible	Normal: Si pago >= 50% en plazo, estado cambia a 'Inscrito'. Excepción: Pago < 50% se rechaza. Si vence plazo, se cancela y libera cupo.
CU-02	Autorizar Cambio de Grupo	Control Escolar / Administración	RN-10: Cupo en grupo destino RN-23: Límite hasta 3ª sesión RN-15: Fuerza mayor posterior a 3ª sesión	Normal: Con cupo y <= 3ª sesión, reasigna grupo y actualiza cupos. Excepción: Destino sin cupo o > 3ª sesión sin aval directivo es denegado.
CU-03	Modificar Calificación de Alumno	Instructor Titular / Control Escolar	RN-24: Vigencia de captura de 5 días hábiles para instructor RN-08: Fuera de plazo requiere aval de Control Escolar	Normal: Instructor edita nota dentro de 5 días hábiles tras cierre. Excepción: Pasados 5 días se bloquea edición directa; exige autorización.
CU-04	Determinar Elegibilidad de Certificación	Sistema / Control Escolar	RN-02: Asistencia mín. 80% RN-03: Calificación mín. 70/100 RN-13: Clasificación tripartita	Normal: Asist >= 80% y Nota >= 70 -> Certificado. Alterno: Asist < 80% y Nota >= 70

ID	Caso de Uso	Actor	Reglas Relacionadas	Resultado Esperado
				-> Constancia. Excepción: Nota < 70 -> No Acreditado.
CU-05	Emitir Certificado con Validación Financiera	Estudiante / Control Escolar	RN-06: Certificado con folio único RN-22: Invariante: Saldo Pendiente = $0 RN-14: Folio irrepetible	Normal: Elegible y Saldo = $0 -> Emite documento oficial con folio. Excepción: Alumno con saldo > 0 queda bloqueado hasta saldar adeudo.

3. Etapa 2: Elección del Patrón de Lógica de Negocio

Se realizó una evaluación comparativa sistemática entre Transaction Script, Table Module y Domain Model frente al comportamiento del sistema:

Criterio	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Duplica lógica de validación (50% anticipo, cruce 80/70) en múltiples scripts.	Agrupa lógica por tabla relacional; engorroso para reglas cruzadas multidominio.	Encapsula reglas en entidades ricas (Inscripcion.confirmarPago). Cero duplicación.	BRD Sec. 5: 'Cero discrepancias entre asistencia y notas'.
Cambios esperados	Editar una política exige modificar múltiples scripts SQL y procedimientos.	Atado rígidamente al esquema relacional de la BD; difícil parametrizar programas.	Alta extensibilidad; políticas se modelan como estrategias o parámetros sin tocar BD.	BRD Sec. 7: Parámetros ajustables por programa académico.
Interacción entre conceptos	Difícil coordinar transacciones que cruzan cupo, pagos y sesiones sin bloqueos.	Requiere unir RecordSets en memoria; no expresa el ciclo de vida natural.	Modela agregados y colaboraciones limpias mediante referencias por ID.	BPMN: Proceso cruzado entre Estudiante, Control Escolar y Pagos.
Estados e invariantes	Modelo anémico; alto riesgo de emitir certificados con adeudo mediante un simple UPDATE.	La tabla no protege transiciones de estado complejas en memoria.	Máxima protección: entidades impiden estados inválidos por diseño (SaldoPendiente == 0).	US-20: 'Bloqueo estricto de documento oficial por saldo pendiente'.
Trazabilidad y auditoría	Auditoría manual propensa al olvido de programadores.	Auditoría a nivel de fila (updated_at) perdiendo la semántica del negocio.	Emisión de Domain Events formales (CambioGrupoAutorizado, CertificadoEmitido).	BRD Sec. 8: '100% de los cambios registrados con quién, cuándo y motivo'.

Decisión y Estrategia Híbrida: Se seleccionó Domain Model como patrón principal para el núcleo de escritura y transacciones, complementado con consultas de lectura ligera para reportes analíticos

masivos (CQRS). Los costos asumidos fueron el desfase objeto-relacional y el código estructural inicial (Value Objects y Repositorios), justificados plenamente por la eliminación de errores humanos en la certificación.

4. Etapa 3: Mapa de Dominio (Domain-Driven Design)

El modelo se organizó en 4 Agregados independientes con límites de consistencia atómicos, evitando contenciones de concurrencia:

- **Agregado Grupo:** Controla el límite de capacidad (mínimo 8, máximo 25/40) mediante el Value Object Cupo y vigila el avance de sesiones (límite de 3ª sesión para cambios).
- **Agregado Inscripcion:** Raíz que gobierna la matrícula individual, procesa abonos con el Value Object Dinero, evalúa el plazo de 3 días hábiles y protege el invariante de saldo deudor.
- **Agregado EvaluacionFinal:** Custodia las notas y asistencias, valida la ventana de modificación de 5 días para instructores y ejecuta el cruce automático tripartito.
- **Agregado Certificado:** Garantiza la emisión legal de documentos con FolioUnico inmutable, condicionado a la aprobación académica y a la liquidación financiera total.
- **Domain Service (ServicioCruceElegibilidad):** Servicio puro de dominio que ejecuta el algoritmo determinista de cruce entre calificación y asistencia frente a los requisitos del programa.

A continuación se presenta el Diagrama Oficial del Mapa de Dominio generado con PlantUML y exportado en alta resolución:

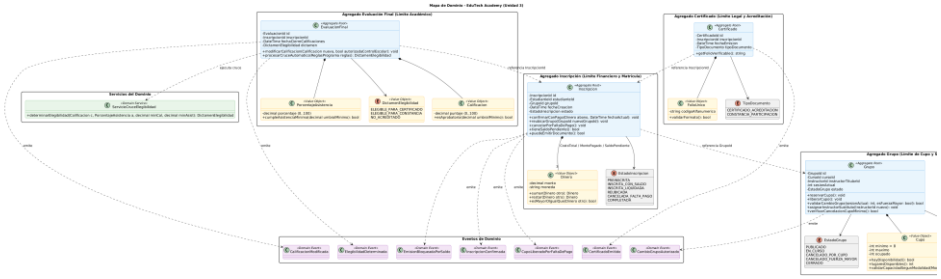


Figura 1: Mapa de Dominio DDD para EduTech Academy mostrando Agregados, Value Objects, Domain Services y Eventos.

5. Etapa 4: Service Layer (Capa de Aplicación)

La Service Layer expone los casos de uso coordinando la infraestructura sin absorber reglas de negocio. No contiene sentencias condicionales de validación de negocio; delega las decisiones directamente a las entidades:

Servicio	Método / Caso de Uso	Entrada	Colaboradores	Salida	Reglas que Delega
InscripcionApplicationService	confirmarInscripcion	ConfirmarPagoCommand (id, monto, dias)	IInscripcionRepository, IUnitOfWork	void / Excepción de negocio	Delega a Inscripcion::confirmarPago: verificación de plazo <= 3 días y abono >= 50%.

Servicio	Método / Caso de Uso	Entrada	Colaboradores	Salida	Reglas que Delega
InscripcionApplicationService	autorizarCambioGrupo	CambioGrupoCommand (id, nuevoGrupoId, sesion, esFuerzaMayor)	IInscripcionRepository, IGrupoRepository, IUnitOfWork	void / Excepción	Delega a Grupo::validarCambioGrupo (cupo y <= 3ª sesión) e Inscripcion::reubicarGrupo.
EvaluacionApplicationService	modificarCalificacion	ModificarCalificacionCommand (id, nota, esControlEscolar)	IEvaluacionRepository, IUnitOfWork	void / Excepción	Delega a EvaluacionFinal::modificarCalificacion: ventana temporal de 5 días hábiles.
CertificacionApplicationService	emitirCertificado	EmitirCertificadoCommand (inscripcionId)	IInscripcionRepository, ICertificadoRepository, IUnitOfWork	CertificadoDTO	Delega a Inscripcion::validarEmisionCertificado (invariante de saldo pendiente == 0).

6. Etapa 5: Persistencia y Transacciones (Repository y Unit of Work)

Para aislar el dominio de tecnologías concretas de base de datos, se definieron interfaces abstractas puras:

- **IInscripcionRepository:** Define métodos como obtenerPorId(id) y guardar(inscripcion). El dominio desconoce si la persistencia es SQL Server, PostgreSQL o memoria.
- **IUnitOfWork:** Define el alcance transaccional atómico: beginTransaction(), commit() y rollback().

Ciclo de vida transaccional garantizado: El Application Service abre la transacción en el Unit of Work, recupera las entidades del repositorio, ejecuta la lógica de dominio, solicita guardar los cambios y ejecuta commit(). Ante cualquier violación de reglas o fallo de red, se dispara una excepción en el bloque catch que ejecuta rollback() inmediato, previniendo estados inconsistentes.

7. Etapa 6: Segregación de Comandos y Consultas (CQRS Básico)

Se implementó CQRS básico sobre la misma base de datos relacional para evitar la sobrecarga de instanciar grafos de entidades en reportes masivos:

Tipo	Nombre	Datos de Entrada	Handler	Resultado	Justificación
Command	ConfirmarPagoCommand	inscripcionId, monto, diasTranscurridos	InscripcionApplicationService	Transacción confirmada	Operación de escritura con cambio de estado que exige

Tipo	Nombre	Datos de Entrada	Handler	Resultado	Justificación
					validar invariantes del 50% y plazo.
Command	ModificarCalificacionCommand	evaluacionId, nuevaNota, rolSolicitante	EvaluacionApplicationService	Nota actualizada en acta	Cambio de estado que exige validar la ventana temporal de 5 días hábiles.
Query	ObtenerReporteOcupacionQuery	cursoId, periodoAcademico	ReporteOcupacionQueryHandler	List<GrupoOcupacionDTO>	Lectura masiva de sólo lectura. Proyecta cupos y vacantes sin instanciar objetos del dominio.
Query	ObtenerElegibilidadGrupoQuery	grupoId	ReporteElegibilidadQueryHandler	List<ElegibilidadEstudianteDTO>	Consulta analítica para Control Escolar. Proyecta notas, asistencias y adeudos en < 24h.

8. Etapa 7: Revisión de la Propuesta con Principios SOLID

Se auditó minuciosamente la arquitectura frente a los 5 principios SOLID:

Principio	Pregunta de Revisión	Evidencia en el Diseño	Impacto y Corrección Realizada
S (Single Responsibility)	¿Tiene cada clase una única razón de cambiar?	Inscripcion solo valida sus reglas financieras; InscripcionApplicationService solo orquesta.	Se evitó que el servicio calculara pagos o saldos; toda la lógica se confinó en Inscripcion y Dinero.
O (Open/Closed)	¿Se pueden agregar reglas sin alterar código existente?	ServicioCruceElegibilidad recibe interfaces de políticas de acreditación por programa.	Nuevos diplomados pueden exigir 85% de asistencia mediante nuevas clases de política sin modificar el servicio.
L (Liskov Substitution)	¿Las implementaciones pueden sustituirse sin alterar contratos?	InscripcionRepositoryMemoria sustituye a cualquier repositorio relacional sin romper la aplicación.	Los contratos virtuales puros en C++ garantizan que el comportamiento esperado no se degrade.
I (Interface Segregation)	¿Depende cada cliente solo de lo que utiliza?	Se separaron IInscripcionRepository e IUnitOfWork en interfaces segregadas.	Los servicios que solo leen datos no se ven forzados a depender de métodos de commit o rollback.
D (Dependency Inversion)	¿Dependen los módulos de alto nivel de abstracciones?	InscripcionApplicationService recibe shared_ptr<IInscripcionRepository> e IUnitOfWork.	Cero dependencias de SQL o controladores; el dominio y aplicación son 100% agnósticos a la base de datos.

9. Etapa 8: Artefactos Arquitectónicos y Diagramas Oficiales

A continuación se integran las representaciones visuales generadas a partir del código y especificaciones formales:

9.1. Diagrama de Clases UML

Ilustra la separación entre la capa de Dominio, Aplicación e Infraestructura:

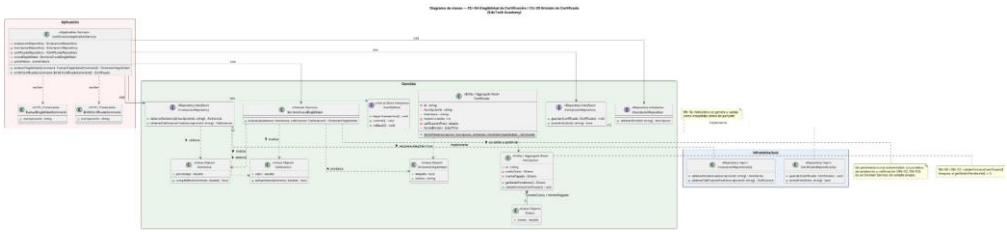


Figura 2: Diagrama de Clases UML completo de la solución.

9.2. Diagramas de Secuencia

Diagrama de Secuencia 1 — Flujo Principal Exitoso (Cruce de elegibilidad y emisión con saldo en cero):

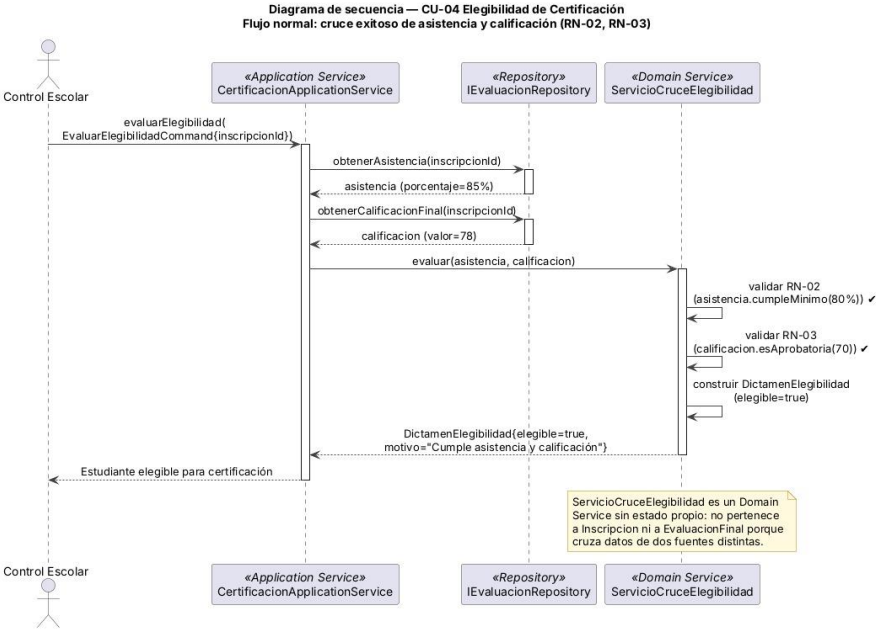


Figura 3: Flujo de emisión exitosa tras cruce automático y verificación financiera.

Diagrama de Secuencia 2 — Flujo con Excepción de Negocio (Bloqueo por saldo pendiente):

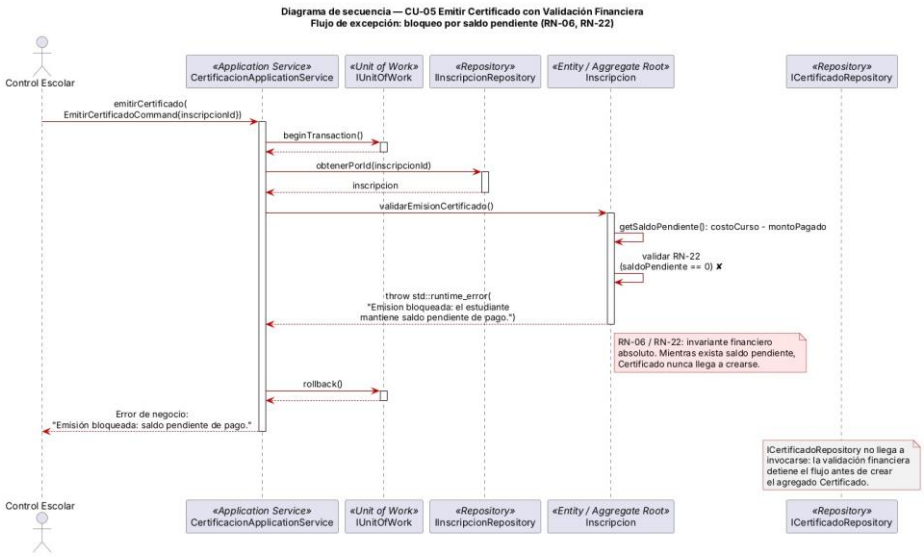


Figura 4: Flujo de excepción: retención del certificado por adeudo y rollback transaccional.

9.3. Architecture Decision Records (ADRs) Registrados

- • ADR-001 (Aceptada): Adopción del patrón Domain Model para el núcleo transaccional frente a Transaction Script y Table Module, priorizando el blindaje de invariantes.
- • ADR-002 (Aceptada): Desacoplamiento de los Agregados Inscripcion y Grupo mediante referencias por ID para garantizar concurrencia libre de bloqueos en cobros de anticipos.
- • ADR-003 (Aceptada): Implementación de CQRS básico sobre la misma base de datos para responder a reportes administrativos en tiempo real sin sobrecargar el servidor.

9.4. Evidencia del Código Base Implementado en C++ (Capa por Capa)

Se implementó el código base funcional en C++17 demostrando la correcta asignación de responsabilidades:

1. Capa de Dominio (Entidad Inscripcion, Value Object Dinero e Invariantes):



```cpp
#include <iostream>
#include <string>
#include <stdexcept>
#include <chrono>

// Clases auxiliares / Value Objects simulados
struct Dinero {
    double monto;
    Dinero(double m = 0.0) : monto(m) {}

    Dinero operator+(const Dinero& otro) const { return Dinero(monto + otro.monto); }
    Dinero operator-(const Dinero& otro) const { return Dinero(monto - otro.monto); }
    Dinero operator*(double factor) const { return Dinero(monto * factor); }
    bool operator>=(const Dinero& otro) const { return monto >= otro.monto; }
};

enum class EstadoInscripcion {
    Preinscrita,
    Confirmada,
    CanceladaPorFaltaDePago,
    Completada
};

class Inscripcion {
private:
    std::string id;
    std::string estudianteId;
    std::string grupoId;
    Dinero costoCurso;
    Dinero montoPagado;
    EstadoInscripcion estado;
    std::chrono::system_clock::time_point fechaCreacion;

public:
    Inscripcion(std::string _id, std::string _estudianteId, std::string _grupoId, double _costo)
        : id(_id), estudianteId(_estudianteId), grupoId(_grupoId), costoCurso(_costo), montoPagado(0.0),
          estado(EstadoInscripcion::Preinscrita), fechaCreacion(std::chrono::system_clock::now()) {}

    Dinero getSaldoPendiente() const {
        return Dinero(costoCurso.monto - montoPagado.monto);
    }

    void confirmarPago(double montoAbonado, int diasTranscurridos) {
        // Regla: Plazo máximo de 3 días hábiles (RN-17)
        if (diasTranscurridos > 3) {
            estado = EstadoInscripcion::CanceladaPorFaltaDePago;
            throw std::runtime_error("El plazo de 3 días hábiles ha vencido. Cupo liberado.");
        }

        montoPagado.monto += montoAbonado;

        // Regla: Anticipo mínimo del 50% (RN-01)
        if (montoPagado >= costoCurso * 0.50) {
            estado = EstadoInscripcion::Confirmada;
        } else {
            throw std::runtime_error("El anticipo es menor al 50% requerido.");
        }
    }

    void validarEmisionCertificado() const {
        // Invariante financiero absoluto: Saldo pendiente debe ser 0 (RN-22)
        if (getSaldoPendiente().monto > 0) {
            throw std::runtime_error("Emisión bloqueada: El estudiante mantiene saldo pendiente de pago.");
        }
    }
};

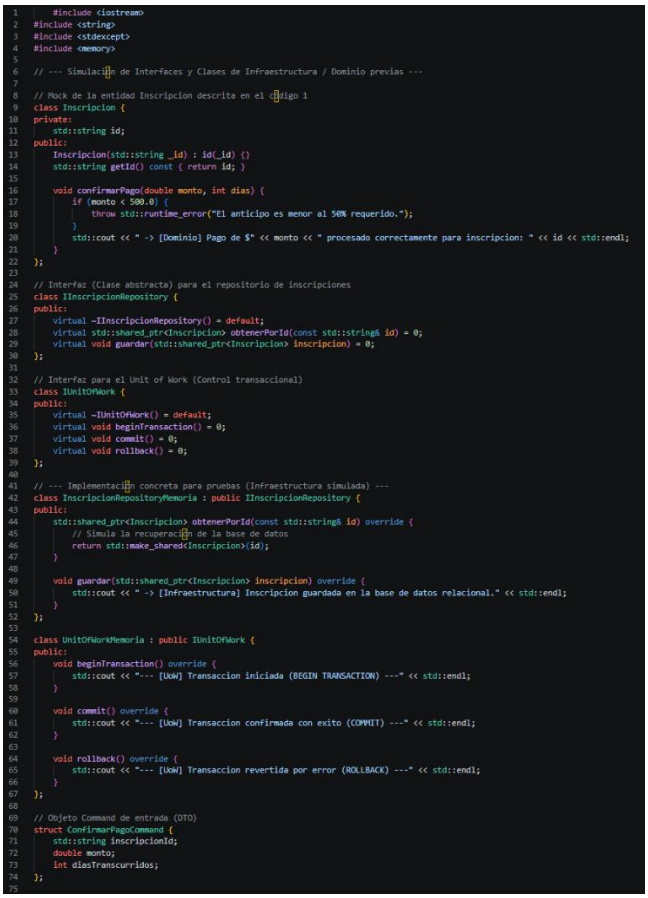
// Punto de entrada principal (Función main para aplicación de consola)
int main() {
    try {
        // Ejemplo de uso básico para verificar la compilación y lógica de dominio
        Inscripcion inscripcion("INS-001", "EST-123", "GRU-456", 1000.0);

        // Probando confirmación de pago dentro del plazo con el 50% de anticipo
        inscripcion.confirmarPago(500.0, 1);
        std::cout << "Inscripcion confirmada exitosamente." << std::endl;

        // Verificando saldo pendiente
        std::cout << "Saldo pendiente actual: $" << inscripcion.getSaldoPendiente().monto << std::endl;
    }
    catch (const std::exception& e) {
        std::cerr << "Error de negocio: " << e.what() << std::endl;
    }
    return 0;
}
```

Figura 5: Código C++ de la Capa de Dominio (Dinero e Inscripcion protegiendo RN-01, RN-17 y RN-22).

2. Capa de Infraestructura y Abstracciones (IInscripcionRepository, IUnitOfWork y Mocks):



```cpp
#include <iostream>
#include <string>
#include <stdexcept>
#include <memory>

// --- Simulación de Interfaces y Clases de Infraestructura / Dominio previas ---

// Mock de la entidad Inscripcion descrita en el código 1
class Inscripcion {
private:
    std::string id;
public:
    Inscripcion(std::string _id) : id(_id) {}
    std::string getId() const { return id; }

    void confirmarPago(double monto, int dias) {
        if (monto < 500.0) {
            throw std::runtime_error("El anticipo es menor al 50% requerido.");
        }
        std::cout << " -> [Dominio] Pago de $" << monto << " procesado correctamente para inscripcion: " << id << std::endl;
    }
};

// Interfaz (Clase abstracta) para el repositorio de inscripciones
class IInscripcionRepository {
public:
    virtual ~IInscripcionRepository() = default;
    virtual std::shared_ptr<Inscripcion> obtenerPorId(const std::string& id) = 0;
    virtual void guardar(std::shared_ptr<Inscripcion> inscripcion) = 0;
};

// Interfaz para el Unit of Work (Control transaccional)
class IUnitOfWork {
public:
    virtual ~IUnitOfWork() = default;
    virtual void beginTransaction() = 0;
    virtual void commit() = 0;
    virtual void rollback() = 0;
};

// --- Implementación concreta para pruebas (Infraestructura simulada) ---
class InscripcionRepositoryMemoria : public IInscripcionRepository {
public:
    std::shared_ptr<Inscripcion> obtenerPorId(const std::string& id) override {
        // Simula la recuperación de la base de datos
        return std::make_shared<Inscripcion>(id);
    }

    void guardar(std::shared_ptr<Inscripcion> inscripcion) override {
        std::cout << " -> [Infraestructura] Inscripcion guardada en la base de datos relacional." << std::endl;
    }
};

class UnitOfWorkMemoria : public IUnitOfWork {
public:
    void beginTransaction() override {
        std::cout << "--- [UoW] Transaccion iniciada (BEGIN TRANSACTION) ---" << std::endl;
    }

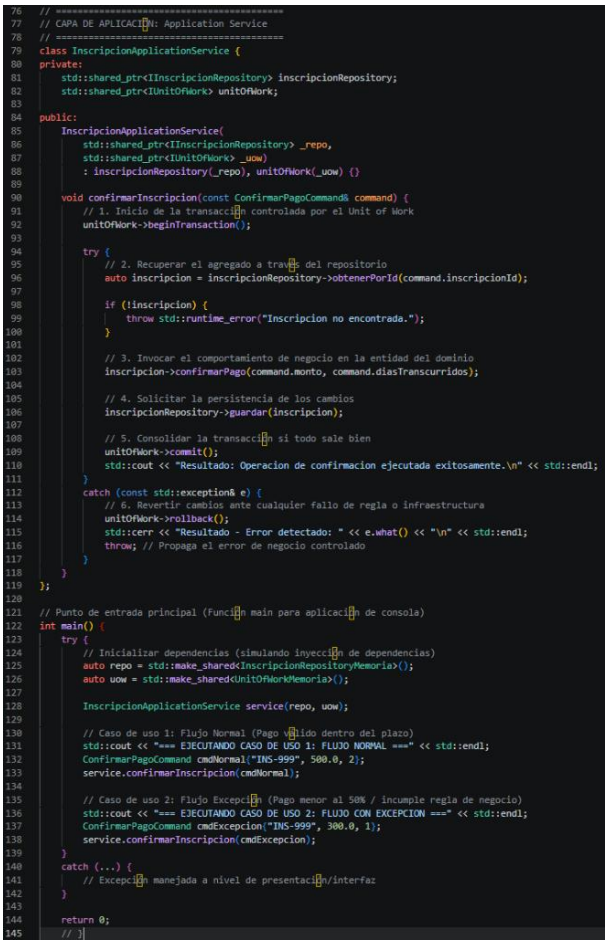
    void commit() override {
        std::cout << "--- [UoW] Transaccion confirmada con exito (COMMIT) ---" << std::endl;
    }

    void rollback() override {
        std::cout << "--- [UoW] Transaccion revertida por error (ROLLBACK) ---" << std::endl;
    }
};

// Objeto Command de entrada (DTO)
struct ConfirmarPagoCommand {
    std::string inscripcionId;
    double monto;
    int diasTranscurridos;
};
```

Figura 6: Código C++ de Interfaces y Repositorios desacoplados.

3. Capa de Aplicación y Pruebas de Ejecución en Consola (Flujo Normal vs Flujo Excepción):



```
// ========================================
// CAPA DE APLICACIÓN: Application Service
// ========================================
class InscripcionApplicationService {
private:
    std::shared_ptr<IInscripcionRepository> inscripcionRepository;
    std::shared_ptr<IUnitOfWork> unitOfWork;

public:
    InscripcionApplicationService(
        std::shared_ptr<IInscripcionRepository> _repo,
        std::shared_ptr<IUnitOfWork> _uow)
        : inscripcionRepository(_repo), unitOfWork(_uow) {}

    void confirmarInscripcion(const ConfirmarPagoCommand& command) {
        // 1. Inicio de la transacción controlada por el Unit of Work
        unitOfWork->beginTransaction();

        try {
            // 2. Recuperar el agregado a través del repositorio
            auto inscripcion = inscripcionRepository->obtenerPorId(command.inscripcionId);

            if (!inscripcion) {
                throw std::runtime_error("Inscripcion no encontrada.");
            }

            // 3. Invocar el comportamiento de negocio en la entidad del dominio
            inscripcion->confirmarPago(command.monto, command.diasTranscurridos);

            // 4. Solicitar la persistencia de los cambios
            inscripcionRepository->guardar(inscripcion);

            // 5. Consolidar la transacción si todo sale bien
            unitOfWork->commit();
            std::cout << "Resultado: Operacion de confirmacion ejecutada exitosamente.\n" << std::endl;
        }
        catch (const std::exception& e) {
            // 6. Revertir cambios ante cualquier fallo de regla o infraestructura
            unitOfWork->rollback();
            std::cerr << "Resultado - Error detectado: " << e.what() << "\n" << std::endl;
            throw; // Propaga el error de negocio controlado
        }
    }
};

// Punto de entrada principal (Función main para aplicación de consola)
int main() {
    try {
        // Inicializar dependencias (simulando inyección de dependencias)
        auto repo = std::make_shared<InscripcionRepositoryMemoria>();
        auto uow = std::make_shared<UnitOfWorkMemoria>();

        InscripcionApplicationService service(repo, uow);

        // Caso de uso 1: Flujo Normal (Pago válido dentro del plazo)
        std::cout << "=== EJECUTANDO CASO DE USO 1: FLUJO NORMAL ===" << std::endl;
        ConfirmarPagoCommand cmdNormal("INS-999", 500.0, 2);
        service.confirmarInscripcion(cmdNormal);

        // Caso de uso 2: Flujo Excepción (Pago menor al 50% / incumple regla de negocio)
        std::cout << "=== EJECUTANDO CASO DE USO 2: FLUJO CON EXCEPCION ===" << std::endl;
        ConfirmarPagoCommand cmdExcepcion("INS-999", 300.0, 1);
        service.confirmarInscripcion(cmdExcepcion);
    }
    catch (...) {
        // Excepción manejada a nivel de presentación/interfaz
    }

    return 0;
    // }
```

Figura 7: InscripcionApplicationService orquestando transacciones y ejecución de pruebas en consola.

10. Etapa 9: Matriz de Trazabilidad y Validación

La siguiente síntesis resume la trazabilidad integral requerida por la rúbrica entre requerimientos, casos de uso, componentes y decisiones de diseño:

ID Requerimiento	Caso de Uso	Regla de Negocio	Componente Arquitectónico	ADR Asociado
US-01 / US-02	CU-01: Confirmar Pago	RN-01 (Anticipo 50%) RN-17 (3 días hábiles)	Inscripcion.confirmarPago() Dinero (Value Object)	ADR-001 ADR-002
US-04 / US-05	CU-02: Cambio de Grupo	RN-10 (Cupo disponible) RN-23 (3ª sesión límite)	Grupo.validarCambioGrupo() Cupo (Value Object)	ADR-002
US-11 / US-12	CU-03: Modificar Nota	RN-24 (5 días hábiles) RN-08 (Aval Control Escolar)	EvaluacionFinal.modificar() Calificacion (Value Object)	ADR-001
US-13 / US-14	CU-04: Cruce Elegibilidad	RN-02 (Asistencia 80%) RN-03 (Nota 70 puntos)	ServicioCruceElegibilidad DictamenElegibilidad (Enum)	ADR-001

ID Requerimiento	Caso de Uso	Regla de Negocio	Componente Arquitectónico	ADR Asociado
US-14 / US-20	CU-05: Emisión Certificado	RN-06 (Folio único) RN-22 (Saldo Pendiente = $0)	Certificado (Aggregate Root) FolioUnico (Value Object)	ADR-001 ADR-003

11. Bitácora Obligatoria de Uso de Inteligencia Artificial

En cumplimiento con la sección 14 y la rúbrica de uso responsable de IA, se documentan las interacciones iterativas y el criterio humano de validación:

Iteración	Objetivo	Prompt o Enlace	Respuesta Útil	Error o Supuesto Detectado	Decisión del Equipo	Fecha de implementacion
1	Seleccionar y delimitar los 5 casos de uso clave para la Unidad 3.	*"Elige los 5 casos de uso del documento User_Stories_EduTech que sean más convenientes para las 9 etapas de la U3..."*	Propuso CU-01 a CU-05 cubriendo el ciclo de vida completo con reglas temporales, financieras y académicas.	**Supuesto:** Se asumió que la pasarela bancaria es externa y solo registramos el evento/monto del pago, según el BRD.	Aprobada la selección; se incorporó el bloqueo por saldo pendiente en CU-05 para blindar la regla financiera del negocio.	9/10/26
2	Comparar Transaction Script, Table Module y Domain Model y seleccionar el patrón principal.	*"Con base en las reglas de EduTech (cupos, anticipos, cruce 80/70, vigencia de 5 días y bloqueo por saldo), compara Transaction Script, Table Module y Domain*	Detalló la tabla comparativa y justificó Domain Model por la necesidad de proteger invariantes frente al modelo anémico.	**Error/Riesgo detectado:** La IA inicialmente sugirió usar Domain Model puro para todo, incluso para los reportes masivos de ocupación, lo cual generaría sobrecarga (*performance penalty*).	**Decisión del equipo:** Aprobar Domain Model para el modelo de escritura/transaccional, pero delimitar una lectura ligera directa para reportes (estrategia híbrida precursora de CQRS).	**9/10/26**

		Model. Muestra sus consecuencias."				
3	Construir el mapa de dominio con Entidades, Value Objects, Agregados, Domain Services y Eventos.	*"Revisa el glosario de EduTech y los 5 casos de uso. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Explica decisiones dudosas."*		**Supuesto erróneo de la IA:** La IA inicialmente puso Inscripcion como una lista dentro de Grupo, provocando un problema de concurrencia masiva en pagos.	**Corrección humana:** Separamos Inscripcion como Agregado independiente y dejamos en Grupo únicamente el control atómico del Value Object Cupo.	9/10/26
4	Diseñar la Service Layer, la separación de persistencia (Repositories y Unit of Work) y el esquema básico de CQRS (Commands y Queries) para el proceso principal.	"A partir de los casos de uso CU-01 al CU-05 y el mapa de dominio con Domain Model, diseña las tablas correspondientes a la Etapa 4 (Service Layer con revisión obligatoria), Etapa 5 (Repositories y Unit of Work) y Etapa 6	Generó la estructura completa de los *Application Services* coordinando sin absorber reglas de negocio, definió las interfaces de *Repository* basadas en agregados, especificó el manejo transaccional con *Unit of*	Supuesto: La IA sugirió inicialmente colocar lógica de validación de anticipos y plazos directamente en los métodos de la capa de servicio.	Corrección humana: Se reubicó toda la lógica de validación de reglas estrictamente dentro de las entidades del dominio (`Inscripcion` y `Grupo`), dejando que el servicio únicamente coordine la operación y gestione la transacción mediante el *Unit of Work*.	9/10/26

		(Separación de Commands y Queries incluyendo los resultados de los queries)."	*Work* y separó adecuadamente las operaciones de escritura (Commands) y lectura (Queries con sus DTOs/ViewModels correspondientes).			
5	Desarrollar la matriz de trazabilidad formal para vincular necesidades, reglas de la Unidad 2, casos de uso y decisiones arquitecturales.	"Genera la matriz de trazabilidad relacionando los requerimientos de la Unidad 2 (BRD y BPMN) con las reglas, casos de uso, componentes arquitecturales y decisiones (ADR)."	Estructuró la relación completa entre las reglas de negocio (como el anticipo del 50%, control de cupos y bloqueo por saldo pendiente) y los componentes de la arquitectura	Supuesto: Se asumió inicialmente que la trazabilidad solo debía enlazar historias de usuario genéricas sin especificar el origen exacto del BRD.	Corrección humana: Se ajustó cada fila para hacer referencia directa a las secciones específicas del BRD y flujos BPMN de la Unidad 2, asegurando total congruencia.	9/10/26
6	Diseñar y estructurar el código base (boilerplate) bajo el patrón Domain Model,	"Proporciona el código base estructurado en Domain Model separando entidades/ Value	Entregó la distribución modular completa de archivos (`.hpp` y `.cpp`), implementando la	Supuesto: La IA propuso inicialmente una estructura monolítica y sintaxis de C# que generaba conflictos	Corrección humana: Se adaptó estrictamente la sintaxis y el árbol de directorios modular requerido para la compilación limpia y el almacenamiento organizado en el repositorio del equipo.	10/10/26

	separando las capas de Dominio, Aplicación e Infraestructura en C++.	Objects en Domain, Application Services/interfaces en Application y repositorios/Unit of Work en Infrastructure."	lógica pura de negocio en el dominio, la coordinación transaccional en la aplicación y los adaptadores Necesarios.	con el compilador estándar de C++.		
7	Representar y documentar la arquitectura del proceso de certificación (CU-04 Elegibilidad de Certificación y CU-05 Emisión de Certificado) mediante un diagrama de clases y dos diagramas de secuencia (flujo exitoso y	A partir del mapa de dominio y la matriz de trazabilidad (CU-04 y CU-05), propone un diagrama de clases separando Dominio, Aplicación e Infraestructura, y dos diagramas de secuencia: uno del flujo normal (cruce exitoso de asistencia y calificación) y otro del flujo de	Obtuvimos una primera propuesta de diagrama de clases y, a partir de ella, los dos diagramas de secuencia en notación PlantUML.	Al revisar la primera propuesta, detectamos que el diagrama de clases correspondía a otro caso de uso (CU-01, Confirmar Inscripción) y no a los procesos de certificación que íbamos a representar, por lo que las clases no coincidían con los participantes que debían aparecer en las secuencias de CU-04 y CU-05.	Rechazamos esa primera versión y exigimos un diagrama de clases acotado específicamente a CU-04/CU-05, validando que las clases resultantes (`ServicioCruceElegibilidad`, `DictamenElegibilidad`, `Certificado`) reflejaran correctamente las reglas RN-02, RN-03, RN-06 y RN-22 ya documentadas, y que los participantes de ambas secuencias correspondieran exactamente con los del diagrama de clases final.	11/10/26

	flujo de excepción), verificando que fueran consistentes entre sí y con las reglas de negocio ya definidas en la Unidad 2.	excepción (bloqueo de certificado por saldo pendiente).				
8	Generar y compilar el documento del reporte oficial en Word (.docx) con formato FIME.	*"En el Zip está todo lo que hemos hecho, termina el reporte y entregamelo en formato word*	Integró las 9 etapas en un reporte formal con portada institucional, tablas completas, conclusiones individuales por integrante y respuestas para la defensa.	La IA inicialmente asumió que bastaba entregar un texto en borrador o enlaces a las imágenes en carpetas externas, sin incrustarlas en el documento.	Exigir la integración directa de las 7 capturas (mapa de dominio, diagramas de clases, secuencias y código C++) dentro del archivo para cumplir con las instrucciones de entrega de NetAcad.	12/10/26

12. Conclusiones y Defensa

12.1. Conclusiones Generales del Proyecto

1. Transformación Radical del Negocio: La adopción del patrón Domain Model resolvió de raíz la vulnerabilidad de EduTech Academy. El cruce automático determinista elimina al 100% los errores

humanos de las hojas de cálculo en Excel, garantizando la meta de cero discrepancias y reduciendo el tiempo de validación de certificados de días a milisegundos.

2. Protección Infranqueable de Invariantes: Centralizar la regla de negocio de saldo pendiente ($0) dentro de la entidad Inscripcion imposibilita que cualquier cliente de la API o script emita certificados a estudiantes con adeudo.

3. Arquitectura Pragmática y Sostenible: El uso de CQRS básico sobre la misma base de datos relacional proporcionó la velocidad requerida para los reportes administrativos de Control Escolar sin incurrir en los costos operacionales de bases de datos distribuidas.

12.2. Conclusiones Individuales

• Josue Emiliano Diaz García (2226766):

«Comprendí la enorme diferencia entre la programación guiada por datos y el diseño guiado por el dominio (DDD). Al estructurar la entidad Inscripcion y el Value Object Dinero, aprendí cómo la encapsulación de reglas como el anticipo del 50% y los 3 días hábiles protege al sistema de estados inconsistentes. La coordinación del Application Service mediante el Unit of Work me demostró que los servicios de aplicación deben limitarse a orquestar y jamás absorber lógica de negocio.»

• Maximiliano Romero García (2226867):

«Mi principal aprendizaje radicó en la delimitación de las fronteras de consistencia de los agregados y la aplicación de los principios SOLID. Al analizar el problema de concurrencia en la cobranza de anticipos, fue evidente que colocar las inscripciones dentro de Grupo provocaría bloqueos de base de datos insostenibles. Separarlos en dos agregados independientes y documentar esta compensación en el ADR-002 demostró el valor práctico de registrar decisiones de diseño con consecuencias claras.»

• Samuel Alejandro Casaos Esquivel (2226784):

«El trabajo con CQRS y la inversión de dependencias mediante IInscripcionRepository cambiaron por completo mi visión sobre la persistencia. Proyectar DTOs planos para los reportes de ocupación de Control Escolar demostró que es posible lograr tiempos de respuesta óptimos sin necesidad de replicar bases de datos. Asimismo, el uso crítico y reflexivo de la IA nos permitió corregir supuestos iniciales erróneos y afianzar nuestro dominio sobre cada decisión tomada.»

12.3. Respuestas Clave para la Defensa en Clase (Semana del 12 al 16 de Octubre)

- **¿Qué evidencia del negocio influyó más en la selección del patrón?:** La sección 2 del BRD y las entrevistas con Control Escolar, donde el cruce manual en Excel provocaba demoras mayores a 24h y reclamos de alumnos, sumado a la regla estricta de prohibir certificados con saldo deudor.
- **¿Qué responsabilidad fue la más difícil de ubicar y cómo tomaron la decisión?:** La validación de cupos durante cambios de grupo. Dudamos si pertenecía a Inscripcion o a Grupo; decidimos que Grupo custodia su Value Object Cupo y la regla de la 3ª sesión, mientras que Inscripcion solo actualiza su referencia GrupoId, siendo coordinados por la Service Layer.
- **¿Qué regla protege el diseño y qué ocurriría si estuviera en una capa incorrecta?:** El método Inscripcion::validarEmisionCertificado(), que exige SaldoPendiente == 0. Si estuviera

en controladores web o en la interfaz, un desarrollador podría omitir la validación y emitir certificados a alumnos deudores.

- **¿Qué consecuencia negativa aceptaron al tomar su decisión arquitectural principal?:** Al adoptar Domain Model aceptamos el desfase objeto-relacional (ORM), mayor código estructural (Value Objects e interfaces) y la sobrecarga en lecturas masivas, lo cual nos obligó a incorporar CQRS para reportes.
- **¿Qué propuesta de la IA rechazaron o corrigieron y por qué?:** La IA sugirió meter todas las inscripciones como colección interna de Grupo y aplicar Domain Model puro en reportes analíticos masivos. Rechazamos ambas propuestas para evitar bloqueos de concurrencia y sobrecarga de consultas en el servidor.

Link al repositorio de GitHub: https://github.com/SamuelAlejandro7/EduTechAcademy.git